



Department of Computer Science and Engineering
Midterm Examination Spring 2024
CSE 321: Operating Systems

Duration: 1 Hour 20 Minutes

Total Marks: 30

Answer the following questions.
Figures in the right margin indicate marks.

-
1. **Discuss** the role and importance of system calls in facilitating interactions between user programs and the operating system. **Provide** an example of a task performed by the operating system and **identify** the system calls required to accomplish it. [2]
- CO1
- Rushan and Shifa, both software developers, collaborated on building an application with a 55% parallel portion. This application employs multiple threads to perform similar computations on a large dataset, a concept they learned during their undergraduate course CSE321.
- b) If the number of cores is reduced from **5** to **3** during application execution, **calculate** and **explain** the change in performance. [2]
- c) **Identify** and **briefly explain** the type of parallelism applicable in this scenario. State the differences between the two types of parallelism. [3]
2. a) **Draw** a Gantt chart and illustrate the execution of the process using the **Round Robin** scheduling algorithm (**time quantum = 3 units**). **Calculate** the **average waiting time** and **number of context switching**. [3+2+1]
- CO1

Processes	Arrival Time	Burst Time
P1	0	4
P2	5	2
P3	9	6
P4	9	8
P5	8	8

- b) Is a time quantum of 3 justified in this scenario? **State** your reasoning. If not, **what** would have been the optimal time quantum? [1]

3. a) Find the output of the following code snippet.

[3]

CO2

```
const int len=3;
static int y1=0;
static int y2=0;
int main(){
    pid_t g;
    int p[]={60,45,5};
    int q=len-1;
    g=fork();
    if(g<0){
        printf("fork failed\n");
    }
    else if(g>0){
        wait();
        y1=y1-(p[0]+p[q]);
        y2=y2-(p[1]-p[q]);
    }
    else{
        y1=y1-(p[0]/p[q]);
        y2=y2-(p[1]*p[q]);
    }
    if(g>0){
        printf("parent is about to terminate\n");
    }
    else{
        printf("child is about to terminate\n");
    }
    printf("y1 = %d\n",y1);
    printf("y2 = %d\n",y2);

    return 0;
}
```

b) **How** do CPU-bound and I/O-bound processes impact system performance, and **how** can the operating system effectively manage them?

[3]

4. In a bank, multiple bank tellers need to access the vault. To ensure limited access at a time, each bank teller is equipped with a security token device that interacts with the vault. When a teller wants to access the vault, they must enter the token into their device to initiate the access request from the vault. The vault's system grants access if the number of tokens currently active is less than the total allocated slots. If occupied, incoming security tokens must wait until another token exits the vault.

CO2

a) **Identify** which synchronization method has been used here? **Which** synchronization problems can be solved by this method?

[3]

b) **Mention** the process synchronization technique that is limited to ensuring synchronization between two processes and briefly **explain** what other drawbacks it has.

[3]

In a system, following conditions are present.

- There are 3 processes: P1, P2 and P3.
- Initialization value of the mutex lock, available=true.
- Ready queue is in the following order, [P2, P3, P1].
- CPU allocation is managed by round robin scheduling algorithm with the time quantum of 8 ms.
- Each statement takes 4 ms to execute.
- Critical section contains 3 statements.
- Remainder section contains 2 statements.

The structure of process P_i in solution using mutex lock:

<pre> acquire() { while(!available) ;//busy wait available=false; } release() { available=true; } </pre>	<pre> do{ acquire(); //critical section release(); //remainder section }while(true); </pre>
---	---

c) **Complete** the table given below for processes P1, P2 and P3 using mutex lock.

[4]

Process 1	Process 2	Process 3
		

